

03. 动态规划

算法设计与分析

核心思想

- ▶ 最优子结构：全局最优解包含子问题的最优解（可递归定义）
- ▶ 重叠子问题：递归求解中大量重复计算相同的子问题
- ▶ 记忆化：存储子问题结果，避免重复计算
- ▶ 无后效性：未来状态只与当前状态有关，与如何到达无关
- ▶ 本质：用空间换时间，将指数复杂度降为多项式

DP 解题框架

- ▶ 1. 定义状态：明确 $dp[i]$ 或 $dp[i][j]$ 的精确含义
- ▶ 2. 状态转移：建立状态之间的递推/递推关系
- ▶ 3. 初始化：确定边界条件（最小子问题的解）
- ▶ 4. 计算顺序：确保计算当前状态时所需子状态已求出
- ▶ 5. 提取答案：从最终状态得到原问题的解
- ▶ 6. 空间优化：分析状态依赖，尝试滚动数组降维

DP 实现方式

【记忆化搜索 (自顶向下)】

```
function dfs(i, j):  
    if memo[i][j] ≠ -1: return memo[i][j]  
    memo[i][j] = 转移计算(dfs(...))  
    return memo[i][j]
```



【递推 (自底向上)】

```
for i = 1 to n:  
    for j = 1 to m:  
        dp[i][j] = 转移计算(dp[i-1][...])
```

经典问题：斐波那契数列

- ▶ 定义： $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$
- ▶ 递归：时间 $O(2^n)$ ，大量重复计算 $F(2)$, $F(3)$...
- ▶ 记忆化：存储已计算值，时间 $O(n)$ ，空间 $O(n)$
- ▶ 递推：从 $F(0)$, $F(1)$ 逐步计算到 $F(n)$
- ▶ 空间优化：只需保存前两项，空间 $O(1)$

$a, b = 0, 1$

for $i = 2$ to n : $a, b = b, a+b$

经典问题：0/1 背包

- ▶ 问题：n 件物品，重量 $w[i]$ ，价值 $v[i]$ ，背包容量 W
- ▶ 状态： $dp[i][w]$ = 前 i 件物品，容量 w 时的最大价值
- ▶ 转移： $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w[i]] + v[i])$
 - 不选第 i 件，或选第 i 件（需 $w \geq w[i]$ ）
- ▶ 时间： $O(nW)$ ，空间： $O(nW)$
- ▶ 空间优化：滚动数组，一维 $dp[w]$ 逆序更新（防止重复选）

经典问题：完全背包

- ▶ 与 0/1 背包区别：每种物品可以选无限次
- ▶ 状态转移： $dp[i][w] = \max(dp[i-1][w], dp[i][w-w[i]] + v[i])$
- ▶ 注意：第二项是 $dp[i]$ 而非 $dp[i-1]$ （可重复选同种物品）
- ▶ 一维优化： $dp[w]$ 正序更新（允许重复）
- ▶ 应用：硬币组合、资源分配、完全背包变体

经典问题：最长公共子序列 (LCS)

- ▶ 问题：求两个序列的最长公共子序列长度
- ▶ 状态： $dp[i][j]$ = 串A前i个与串B前j个的LCS长度
- ▶ 若 $A[i] == B[j]$: $dp[i][j] = dp[i-1][j-1] + 1$
- ▶ 若 $A[i] != B[j]$: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$
- ▶ 时间复杂度： $O(mn)$ ，空间复杂度： $O(mn)$

经典问题：最长递增子序列 (LIS)

- ▶ 问题：求数组中最长严格递增子序列长度
- ▶ 状态： $dp[i]$ = 以第 i 个元素结尾的LIS长度
- ▶ 转移： $dp[i] = \max(dp[j] + 1)$ for all $j < i$ and $a[j] < a[i]$
- ▶ 时间： $O(n^2)$ ，空间： $O(n)$
- ▶ 优化：二分查找维护单调序列，时间 $O(n \log n)$

经典问题：矩阵链乘

- ▶ 问题：给定矩阵链 $A_1 \times A_2 \times \dots \times A_n$ ，求最优计算顺序
- ▶ 状态： $dp[i][j]$ = 计算 $A[i..j]$ 的最小乘法次数
- ▶ 转移： $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + p[i-1] \cdot p[k] \cdot p[j])$
→ 枚举分割点 k , $i \leq k < j$
- ▶ 时间： $O(n^3)$ ，空间： $O(n^2)$
- ▶ 输出：最优括号化方案，需记录分割点

经典问题：编辑距离

- ▶ 问题：将串A转换为串B的最少操作次数（插入/删除/替换）
- ▶ 状态： $dp[i][j]$ = A[1..i] 转 B[1..j] 的最小代价
- ▶ 若 $A[i]==B[j]$ ： $dp[i][j] = dp[i-1][j-1]$
- ▶ 否则： $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$
 - 删除、插入、替换三种操作的最小值
- ▶ 应用：拼写检查、DNA序列比对、版本控制 diff

DP 与分治、贪心的对比

【分治】子问题独立，无重叠，不存储中间结果（归并排序）

【动态规划】子问题重叠，必须存储中间结果，求最优解

【贪心】局部最优即全局最优，无需回溯，不存储所有状态

▶

▶ 选择策略：

→ 能证明最优子结构 + 重叠子问题 → 用 DP

→ 能证明贪心选择性质 → 用贪心（更快）

→ 子问题独立无重叠 → 用分治

DP 常见优化技巧

- ▶ 滚动数组：状态只依赖前一行/前一列时，降维处理
- ▶ 前缀和优化：区间求和时，用前缀和将 $O(n)$ 降为 $O(1)$
- ▶ 单调队列优化：滑动窗口最值，将 $O(nk)$ 降为 $O(n)$
- ▶ 状态压缩：用二进制位表示集合，处理小规模集合 DP
- ▶ 四边形不等式：优化区间 DP 的分割点枚举